

SHIV NADAR UNIVERSITY CHENNAI

Department of Computer Science and Engineering

DEEP LEARNING LABORATORY

Experiment: Perceptron Learning Algorithm

**Implementation of Perceptron Learning Algorithm
for OR, AND and NOT Logic Gates**

Name: Sree Charan Sharma Swayampakula

Programme: B.Tech Artificial Intelligence and Data Science

Course: Deep Learning Laboratory

Experiment: Perceptron Learning Algorithm

Academic Year: 2026–27

Contents

1 Aim	2
2 Objectives	2
3 Theory	2
4 Software and Libraries Used	3
5 OR Gate	3
5.1 OR Gate Weight Updates	3
5.2 OR Gate Decision Boundaries	4
6 AND Gate	6
6.1 AND Gate Weight Updates	6
6.2 AND Gate Decision Boundaries	7
7 NOT Gate	9
7.1 NOT Gate Weight Updates	9
7.2 NOT Gate Decision Boundaries	9
8 Final Results	11
8.1 Prediction Verification.....	11
9 Observations	12
10 Conclusion	12
11 References	12

1 Aim

To implement the Perceptron Learning Algorithm in Python for the OR, AND, and NOT logic gates, display the weights after every weight update, and plot the decision boundary after each weight update.

2 Objectives

The objectives of this experiment are:

1. To understand the working principle of a single-layer perceptron.
2. To implement the Perceptron Learning Algorithm using Python.
3. To train perceptrons for OR, AND, and NOT logic gates.
4. To display the weight values after every update.
5. To visualize the decision boundary after every weight update.
6. To verify the final predictions of the trained perceptrons.

3 Theory

A perceptron is one of the simplest models of an artificial neuron. It is a binary classifier that calculates a weighted sum of its inputs and applies an activation function to produce an output.

For two input features, the weighted sum can be represented as

$$z = w_0 + w_1x_1 + w_2x_2 \quad (1)$$

where:

- w_0 is the bias weight,
- w_1 and w_2 are the input weights,
- x_1 and x_2 are the input features.

A step activation function is used:

$$\hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \quad (2)$$

The error is calculated as

$$e = y - \hat{y} \quad (3)$$

where y is the target output and $\hat{y}$ is the predicted output.

The perceptron weight update rule is

$$w_i^{new} = w_i^{old} + \eta ex_i \quad (4)$$

where η is the learning rate.

For the bias term, the corresponding input is considered as

$$x_0 = 1 \quad (5)$$

Therefore, the complete weight vector can be updated using

$$\mathbf{w}^{new} = \mathbf{w}^{old} + \eta(y - \hat{y})\mathbf{x} \quad (6)$$

The decision boundary is obtained from

$$w_0 + w_1x_1 + w_2x_2 = 0. \quad (7)$$

Hence,

$$x_2 = -\frac{w_0 + w_1x_1}{w_2}. \quad (8)$$

4 Software and Libraries Used

- Python 3
- Google Colab
- NumPy
- Matplotlib

NumPy is used for numerical operations and array manipulation, while Matplotlib is used to visualize the decision boundaries.

5 OR Gate

The truth table for the OR gate is shown below.

Table 1: OR Gate Truth Table

X_1	X_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

The OR gate is linearly separable and can therefore be learned by a single-layer perceptron.

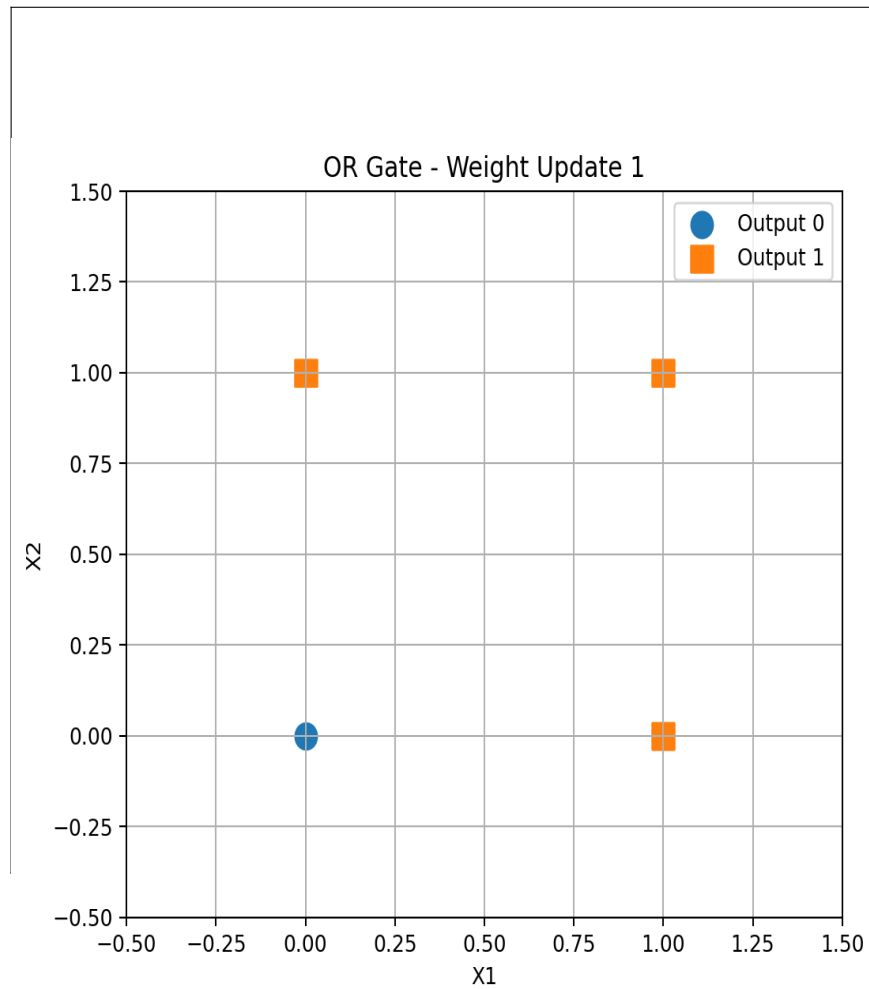
5.1 OR Gate Weight Updates

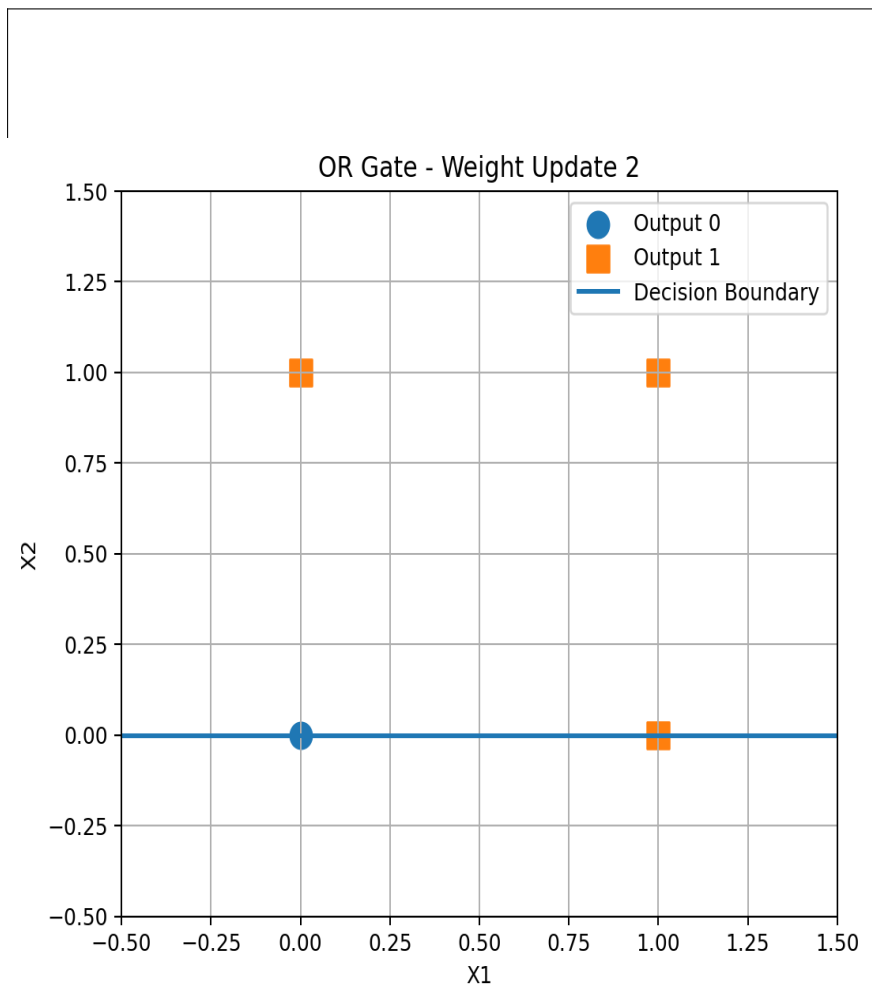
The program initializes the weights to zero and updates them whenever the prediction differs from the target output.

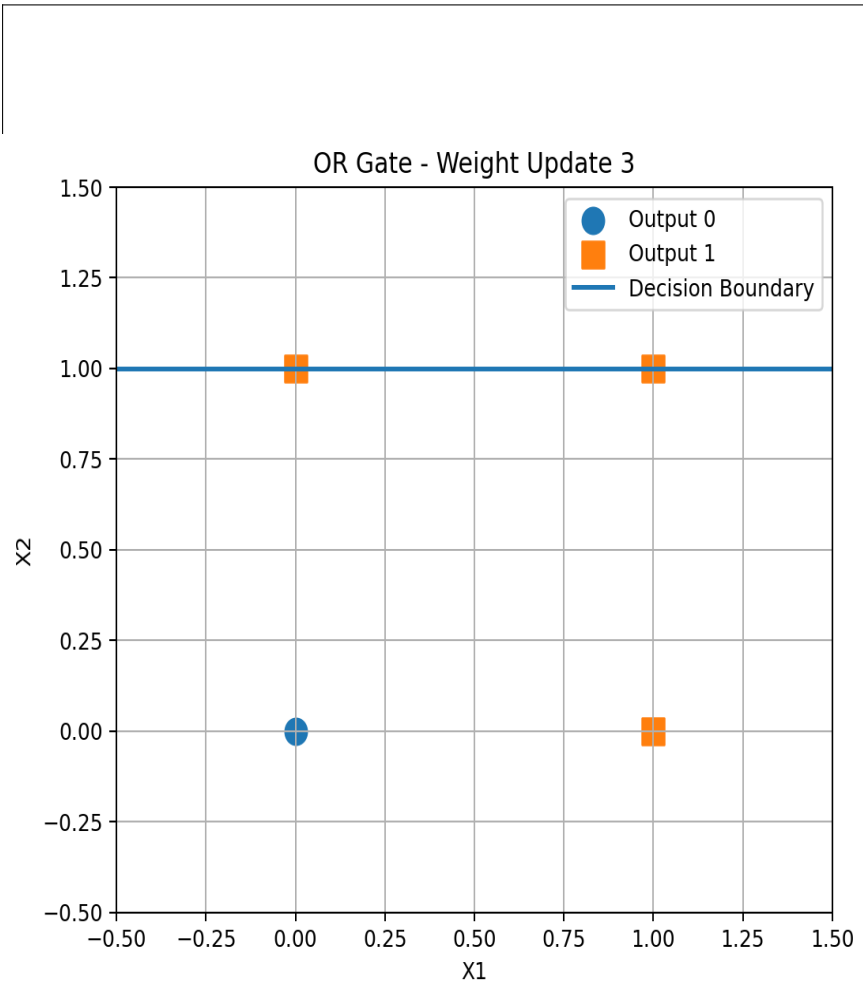
The weights obtained after each update are displayed in the Google Colab output.

5.2 OR Gate Decision Boundaries

The following figures show the decision boundary after each weight update.







6 AND Gate

The truth table for the AND gate is shown below.

Table 2: AND Gate Truth Table

X_1	X_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

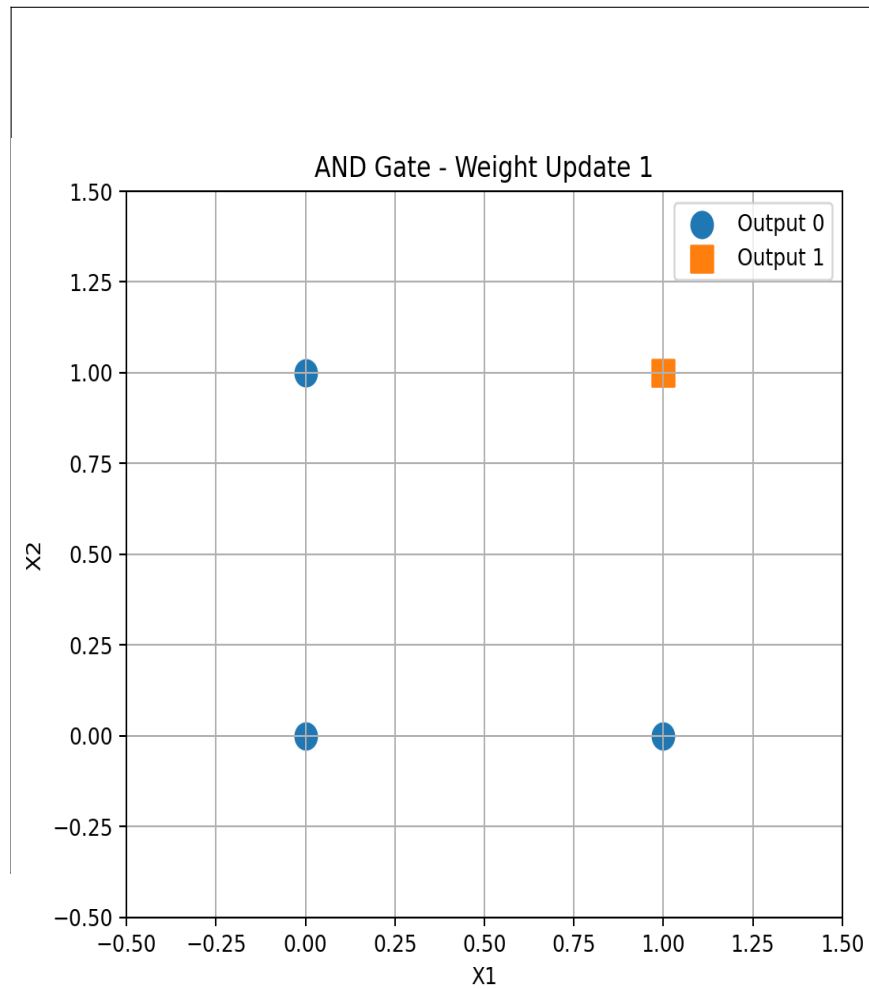
The AND gate is also linearly separable and can be implemented using a single-layer perceptron.

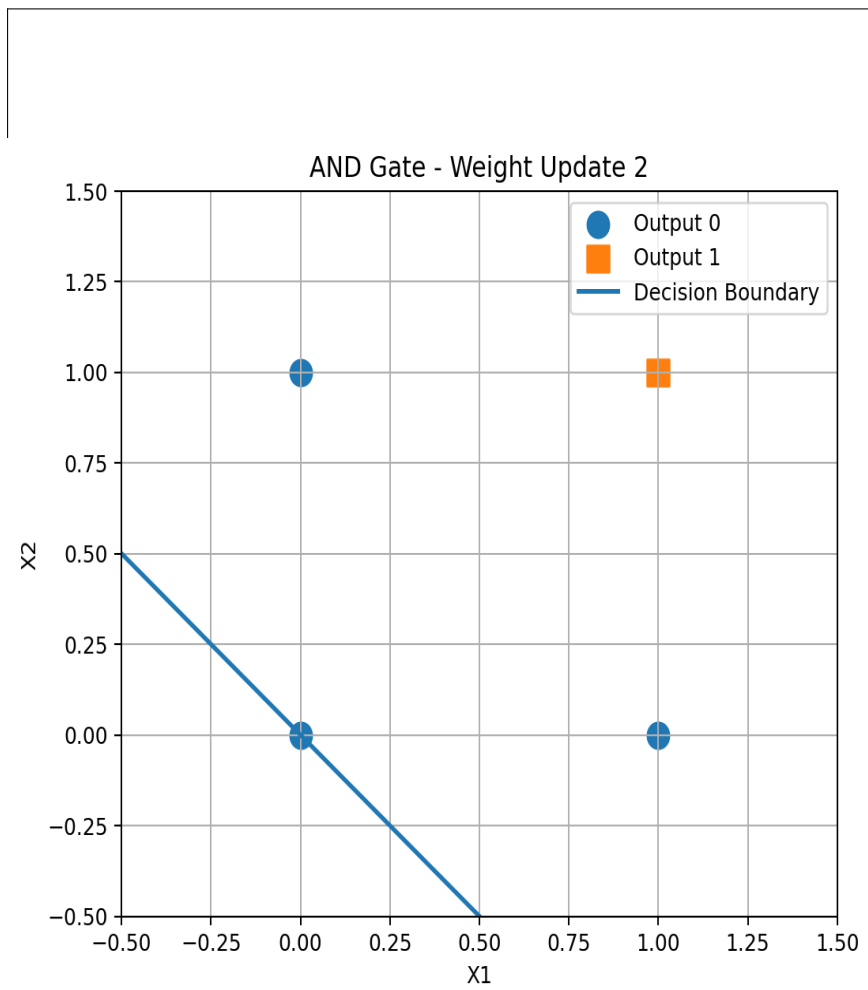
6.1 AND Gate Weight Updates

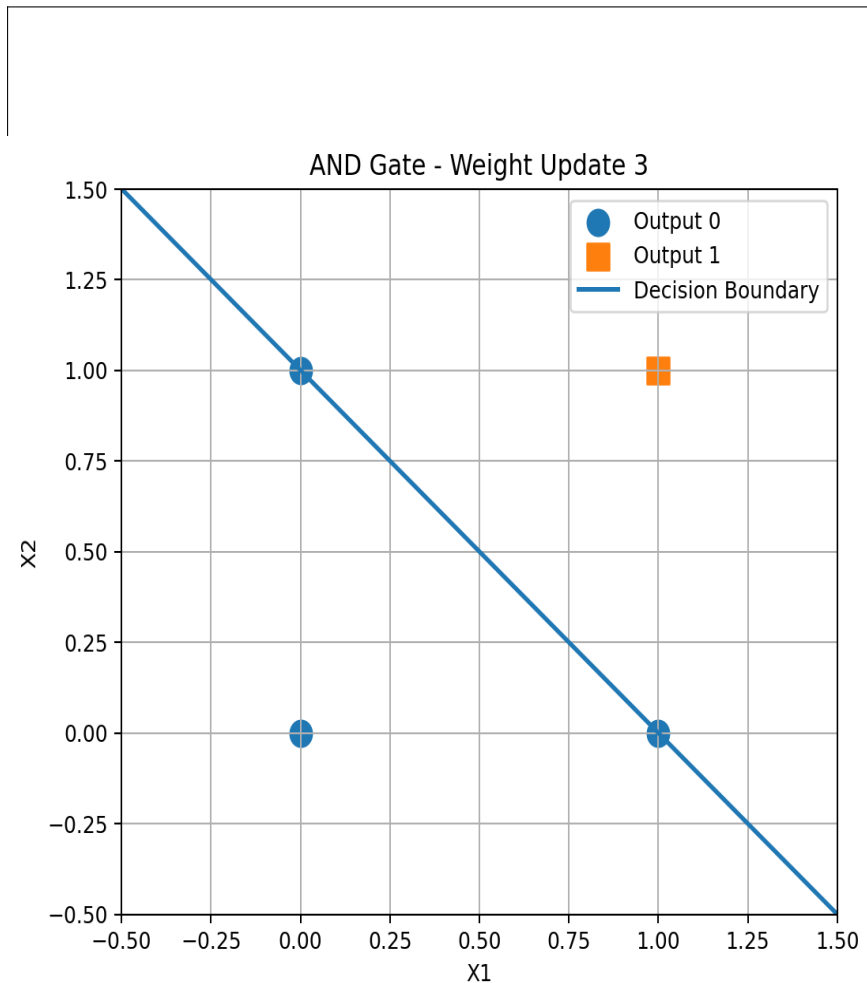
During training, the perceptron updates its weights whenever an incorrect prediction is obtained. The weight values after every update are displayed in the program output.

6.2 AND Gate Decision Boundaries

The following figures show the decision boundary after each weight update.







7 NOT Gate

The NOT gate has a single input.

Table 3: NOT Gate Truth Table

Input	Output
0	1
1	0

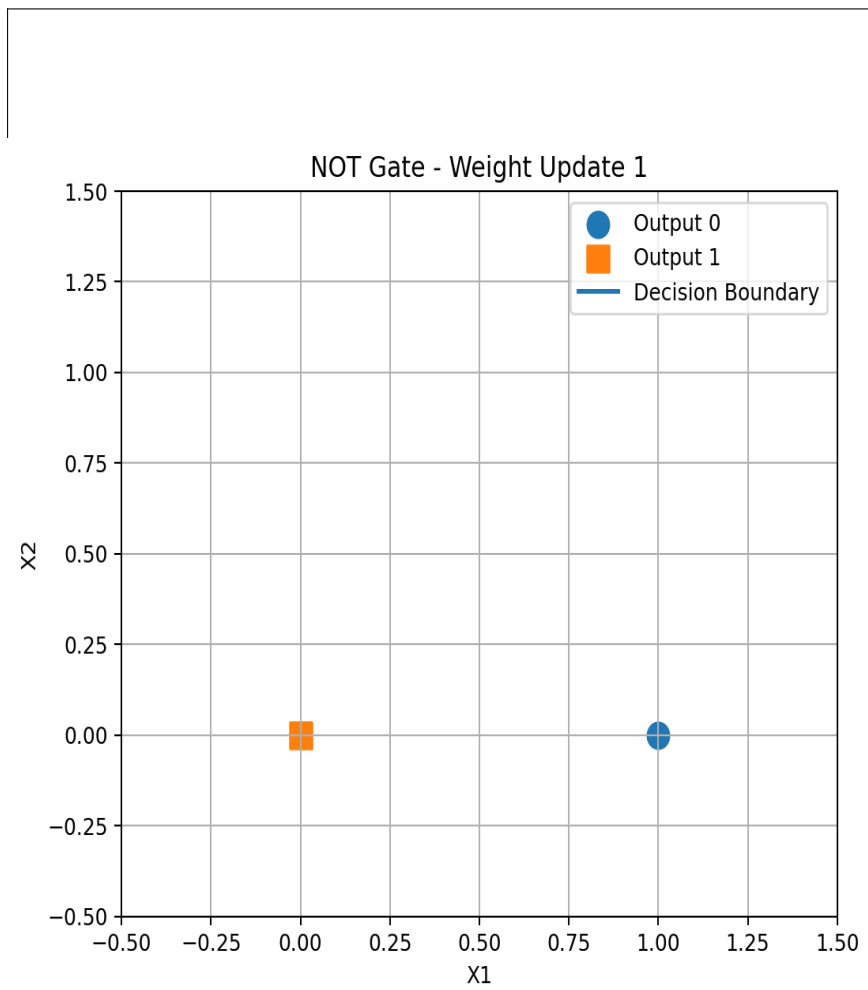
The NOT gate is a one-input binary logic gate. For compatibility with the two-dimensional visualization used for the OR and AND gates, the implementation can represent the second feature as a dummy feature with value zero.

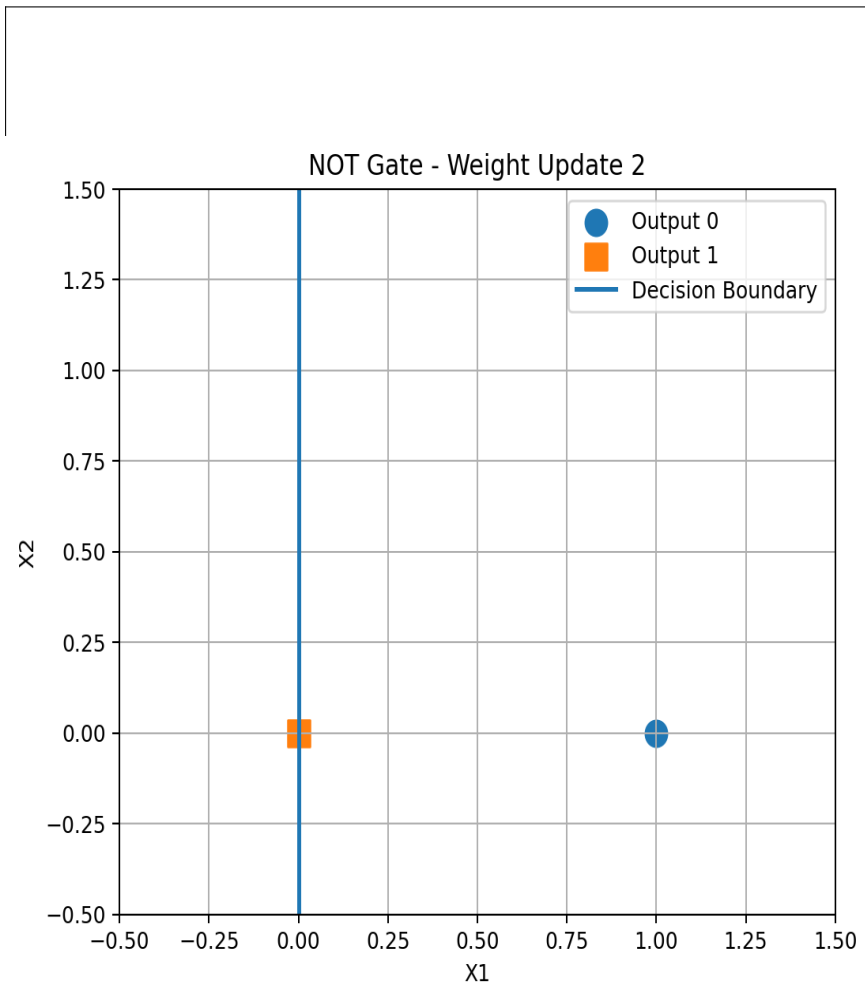
7.1 NOT Gate Weight Updates

The perceptron updates the weights whenever the predicted output differs from the target output. The weights after each update are displayed in the Colab output.

7.2 NOT Gate Decision Boundaries

The following figures show the decision boundary generated during training.





8 Final Results

After training, the final weight vectors are obtained for all three logic gates.

Table 4: Final Perceptron Weights

Gate	Bias (w_0)	w_1	w_2
OR	-0.1	0.1	0.1
AND	-0.1	0.1	0.1
NOT	0.0	-0.1	0.0

The actual numerical values should be filled using the final weights printed by the Google Colab implementation.

8.1 Prediction Verification

The trained perceptrons are evaluated using their corresponding truth tables.

Table 5: Prediction Verification

Gate	Actual Output	Predicted Output	Status
OR	[0,1,1,1]	[0,1,1,1]	Correct
AND	[0,0,0,1]	[0,0,0,1]	Correct
NOT	[1,0]	[1,0]	Correct

9 Observations

1. The perceptron starts with all weights initialized to zero.
2. A weight update occurs when the predicted output does not match the target output.
3. The weight vector changes progressively during training.
4. The decision boundary changes after each weight update.
5. The OR and AND gates are linearly separable and can be learned using a single-layer perceptron.
6. The NOT gate can also be represented by a single-input perceptron.
7. Once the perceptron makes no classification errors during an epoch, the training process converges.

10 Conclusion

The Perceptron Learning Algorithm was successfully implemented in Python for the OR, AND, and NOT logic gates. The implementation calculated predictions using a step activation function and updated the weights whenever a classification error occurred.

The weights after every update were displayed, and the corresponding decision boundary was plotted after each weight update. The final predictions were verified against the truth tables of the three logic gates.

The experiment demonstrates how a single-layer perceptron learns linearly separable binary classification problems by iteratively adjusting its weights.

11 References

1. Deep Learning Laboratory experiment material.
2. Python NumPy documentation.
3. Matplotlib documentation.